

Plan: CSR-Aligned Sign Lookup for top- K Cycle Enumeration

2026-05-10

Scope and goal

Replace the `HashMap<(u32, u32), i8>` sign lookup table used inside `hymeko_graph::topk_cycles`'s DFS routines with a CSR-aligned `Vec<i8>` parallel to the existing `col_idx` array.

Profile (`cargo flamegraph`, Epinions, $k=4$, $m=128$, balance pruner; baseline 114.9s wall) attributes $\sim 39\%$ of CPU cycles to `BuildHasher::hash_one` and `DefaultHasher::write` — both inside the per-edge sign lookup. Replacing the hash-keyed lookup with an array index eliminates that overhead by construction. No algorithmic change; same cycles enumerated in the same order.

Goal: bring Epinions $k=4$, $m=128$ wall time below the **75 s budget** (35% reduction floor; conservative target). Goal must be confirmed by a post-fix flamegraph SVG attached to the report (CLAUDE.md §3 “performance regression discipline”).

Affected files

- `hymeko_graph/src/signed_graph.rs` — additive: a new method `build_csr_with_signs()` returning `(Vec<u32>, Vec<u32>, Vec<i8>)`. Existing `build_csr` and `build_sign_lookup` unchanged.
- `hymeko_graph/src/topk_cycles.rs` — replace the `sign_lookup: &HashMap<(u32, u32), i8>` parameter on the four DFS helpers with `signs_csr: &[i8]`, and update the four enumeration entry points (`enumerate_top_k_cycles`, `enumerate_top_k_per_vertex_cycles`, `*_par` variants) to use the new helper.
- `hymeko_graph/tests/csr_sign_lookup.rs` — new integration test: parity check between the old HashMap-based path and the new CSR-array path on a fixed signed graph fixture.
- `hymeko_graph/benches/topk_balance.rs` — new Criterion benchmark: $k=4$ on a 5k-vertex synthetic Erdős–Rényi signed graph, asserts the post-fix median wall time is at most $0.75\times$ the pre-fix baseline (recorded in `baseline.json` alongside the bench).
- `hymeko_graph/examples/profile_topk_balance.rs` — already created. Used to capture before/after flamegraphs.

CORE.YAML items touched

None. `hymeko_graph` is not listed under `crates`, `files`, or `globs` in `CORE.YAML`. `tests/**` and `benches/**` are in the `allowlist`. No new dependencies (`criterion` is the canonical Rust benchmarking tool per CLAUDE.md §10 and is already a dev-dependency in this workspace; verified during plan).

Interface changes

Internal helpers only — no public-API or PyO3 binding change. The PyO3 wrapper `enumerate_top_k_per_vertex` keeps its signature unchanged.

New public-on-the-crate (but `pub(crate)` or `pub` for test access) method on `SignedGraph`:

```
impl SignedGraph {
    /// Build CSR adjacency along with a parallel per-edge sign
    /// array. 'signs_csr[k]' is the sign of the directed edge
    /// '(start_of_row(k) -> col_idx[k])'.
    ///
    /// # Postconditions
    /// - 'row_ptr.len() == n_nodes + 1'
    /// - 'col_idx.len() == signs_csr.len()'
    /// - For every undirected edge '(u, v, s)' in 'self.edges',
    ///   both '(u, v, s)' and '(v, u, s)' appear in the CSR.
    pub fn build_csr_with_signs(&self)
        -> (Vec<u32>, Vec<u32>, Vec<i8>);
}
```

DFS helper signature change (private):

```
// before
fn dfs_per_vertex<P, S>(
    start: u32,
    row_ptr: &[u32],
    col_idx: &[u32],
    sign_lookup: &HashMap<(u32, u32), i8>,
    ...
)

// after
fn dfs_per_vertex<P, S>(
    start: u32,
    row_ptr: &[u32],
    col_idx: &[u32],
    signs_csr: &[i8],
    ...
)
```

The signed-edge lookup at cycle closure changes from a HashMap query to a CSR scan:

```
// before
let key = (last.min(start), last.max(start));
sign_lookup.get(&key).expect("edge present")

// after
let s = row_ptr[u as usize] as usize;
let e = row_ptr[u as usize + 1] as usize;
let pos = col_idx[s..e].iter().position(|&x| x == v)
    .expect("edge present");
signs_csr[s + pos]
```

Average degree on the workload graphs is small ($\bar{d}_{\text{Epinions}} \approx 13$); the linear scan replaces a SipHash-based HashMap hit. The per-edge cost drops from ~ 70 ns to ~ 5 – 10 ns at typical cache locality.

Test strategy

Unit (hymeko_graph crate, cargo test --release)

- `signed_graph::tests::csr_with_signs_postcondition`: `build_csr_with_signs()` satisfies the documented postconditions on a 6-vertex hand-checked fixture.
- `signed_graph::tests::csr_with_signs_matches_hashmap`: for every (u, v) entry in `build_sign_lookup()` the CSR-indexed lookup yields the same sign. Same fixture.
- Existing `topk_cycles` unit tests must keep passing unchanged — none reference the internal `sign_lookup` parameter.

Integration (hymeko_graph/tests/csr_sign_lookup.rs)

- Generate a 200-vertex Erdős–Rényi signed graph (seed 42, edge probability 0.05, sign probability 0.7 positive).
- Run `enumerate_top_k_per_vertex_cycles_par` with `CartwrightHararyPruner(OnlyBalanced)`, $k = 4$, $m = 8$.
- Assert the cycle set (canonicalised by sorted tuple) is identical to the result obtained from the equivalent HashMap-path fixture run before the change.
- Run again with `NoOpPruner` and the `fraction_negative` scorer; same parity assertion.

Performance (Criterion bench, cargo bench)

- Bench: `topk_balance/erdos_renyi/k4_m32`, 5 000-vertex seeded random graph (deterministic, fixture-hashed).
- Per CLAUDE.md §3 “benchmark stability”: minimum 5 iterations after warm-up, report median, IQR, worst case.
- **Numerical budget assertion**: $\text{post-fix median} \leq 0.75 \times \text{pre-fix median}$ (recorded as `benches/baseline.json`). A regression $> 10\%$ blocks completion per CLAUDE.md §3.
- Peak RSS via `dhat` feature flag (§10 canonical Rust memory profiler): asserted to drop by at least the size of the `sign_lookup` HashMap that is no longer allocated ($\sim 24 \text{ B} \times |E|$).

Pre-/post-fix flamegraph

Both flamegraphs (`target/profile/topk_balance*_pre.svg`, `*_post.svg`) attached to the report. The post-fix flamegraph must show `BuildHasher::hash_one` below 1% of total cycles, or the speedup attribution is invalid (§3 “a $> 10\%$ improvement must be confirmed by profile, not by single-shot wall-clock variance”).

Performance budget

Metric	Pre (measured)	Post (budget)	Source
Epinions $k=4$, $m=128$ wall time	114.9 s	$\leq 75 \text{ s}$	flamegraph (k=4 m=128)
<code>hash_one</code> share of cycles	$\sim 39\%$	$< 1\%$	flamegraph attribution
Allocator share of cycles	$\sim 12\%$	$\leq 8\%$	no HashMap rehashing
Peak RSS, Epinions enum	— (re-measure)	$\leq \text{pre} - 24 E $	<code>dhat</code>
Cycle output (parity)	3 692 194	3 692 194 (exact)	integration test

The 16 GB hard cap (§4) is far above measured peak; this change reduces RSS, does not lift it. No new GPU memory.

Rollback path

The change is one Rust file (`topk_cycles.rs`) plus an additive helper on `SignedGraph`. Single revert restores the `HashMap` path:

```
git revert <commit-sha>
cargo test --release # all green pre-change
```

The PyO3 binding signature is unchanged, so a revert does not require re-publishing the wheel. No external-API surface affected.

If the post-fix flamegraph fails to demonstrate `hash_one` < 1% (i.e., the bottleneck moved somewhere unexpected), the change is held under review per §3 — not merged on wall-clock alone.

Architecture diagram

See `plan.tikz` (rendered below) for the data-layout change. See `plan.mmd` for the call-sequence diagram.

